

Problem 1

The *score* of a sequence of integers $b_1, b_2, \dots, b_\ell$ is defined as

$$\sum_{1 \leq i < \ell} (b_i - b_{i+1})^2.$$

Describe and analyze an $O(n^2k)$ -time dynamic programming algorithm to compute the highest possible score over all subsequences of length at most k in a given array $A[1..n]$ of integers most k , where k is a given parameter. For example, 1, 9, 3 is a subsequence of $[3, \underline{1}, 5, 6, \underline{9}, 7, 4, \underline{3}, 8]$ of length 3 with score $(1 - 9)^2 + (9 - 3)^2 = 64 + 36 = 100$, which is the highest-scoring 3-length subsequence in the array. Note that the items in a subsequence need not be contiguous in the original array.

Solution:

$\text{MaxScore}(A[1..j], l)$: $\text{MaxScore}(A[i..j], l)$ returns the maximum score over all subsequence of length at most l in given array $A[1..j]$.

Input:

$$\text{MaxScore}(A[1..j], l) = \begin{cases} 0 & \text{if } l = 1 \text{ or } i = j \\ (A[1] - A[j])^2 & \text{if } j - 1 = 1 \\ \max_{1 < g \leq j} \{ (A[1] - A[g])^2 + \text{MaxScore}(A[g..j], l - 1) \} & \text{otherwise} \end{cases}$$

Memoization DS: See that our parameters only depend on n and k . Hence, a 2D array of size $n \times k$ suffice.

Evaluation Order: We need to fill the memoization table bottom-up and left-right.

Initialize base cases ($l=1$ and $j=1$) Iterate l from 2 to k For each l , iterate j from 2 to n For each (l, j) pair, compute $\text{MaxScore}(A[1..j], l)$ using the recurrence relation

Runtime Analysis:

We have a 2D table of size $n \times k$ to fill. For each cell, we're taking a maximum over at most n values. Therefore, the time complexity is $O(n * k) * O(n) = O(n^2k)$.

Solving the Problem: To solve the original problem, we compute $\text{MaxScore}(A[1..n], k)$. Then return indices $\text{Memo}[n][k]$ from the memoization table.

Problem 2

The sequel to the worldwide classic board game *Snakes and Ladders* is here: *Just Ladders*! The game of *Just Ladders* is not a race to the finish nor does it involve any randomness by way of dice rolling or spinner spinning; instead, it is a game about collecting as many points as possible by the time your token reaches the end of the board.

A game of *Just Ladders* consists of a $\sqrt{d} \times \sqrt{d}$ board, where $k > 0$ is a perfect square, that has $n \geq 1$ “ladders.” The spaces are numbered so that there is a path along adjacent spaces starting from space 1 to space k , and each space i may have at most one ladder leading up from that space to a higher space j , $j \geq i$. A ladder leading up from a space i is associated with a number of points, $p_i \geq 0$. Note that a space may have many ladders leading into it but at most one leading up from it.

As a player of the game, you begin with your token on space 1. Then you repeat the following until your token reaches the last space on the board, space n :

1. You can always move your token from its current space i to a later space $j > i$, for no gain in points.
2. Additionally, if your current space i has a ladder *leading up* from that space to another space $k > i$, you can use the ladder to move your token to space k and gain p_i points for using it.

Formally, a board for *Just Ladders* consists of a perfect square integer k , denoting the size of the board, and three arrays $B[1..n]$, $T[1..n]$, and $P[1..n]$, that represent the $n \geq 1$ ladders: B is a sorted (potentially non-contiguous) subsequence of $[1, 2, \dots, n]$ such that there is a ladder leading up from space $B[i]$ to space $T[i]$, $B[i] < T[i]$, that awards $P[i]$ points for using the ladder, for each i from 1 to k .

Design and analyze an efficient dynamic programming to compute the maximum number of points that is possible to obtain given board size d and arrays B , T , and P . The target runtime is $O(n \log n)$. [Hint: Don't aim for the target runtime from the start. First obtain a **correct** solution, and **then** consider how to improve the runtime.]

Solution: MaxPts(i): For input index i , $1 \leq i \leq n$, returns the maximum number of points given $B[1..n]$, $T[1..n]$, and $P[1..n]$.

We first define a LeastGreatestElement($T[i]$, B) function, which returns the index of the least greatest value of $T[i]$ in B , where B is sorted.

```
LGE(target, B[1..n]):
  left ← 1
  right ← n
  result ← n + 1
  while left ≤ right:
    mid ← ⌊(left + right)/2⌋
    if B[mid] ≥ target:
      result ← mid
      right ← mid - 1
    else:
      left ← mid + 1
  return result
```

$$\text{MaxPts}(i) = \begin{cases} 0 & \text{if } i > n \\ p[i] & \text{if } T[i] > B[n] \\ \max \left\{ \begin{array}{l} p[i] + \text{MaxPts}(j) \text{ where } j = \text{LGE}(T[j], B[1..n]) \\ \text{MaxPts}(i + 1) \end{array} \right\} & \text{otherwise} \end{cases}$$

Memoization DS: We can use a 1D array `memo[1..n]` to store the maximum points for each ladder index.

Evaluation Order: We should evaluate from right to left, starting from `n` down to `1`, because the recurrence relation depends on future states.

Runtime Analysis: In the initial solution: We have `n` subproblems to solve. For each subproblem, we are looking for the least greatest element, which is $O(\log n)$. This gives us an time complexity of $O(n \log n)$.

Solving the Problem: To solve the original problem, we compute `MaxPts(1)`. And compute `memo[n]`.

Problem 3

The corporate structure at $\mathbb{Y}$, a recently-renamed social media company, can be represented by a rooted tree, where the employees are nodes and the child nodes of an employee are other employees that directly report to them. The CEO, at the root of the tree, does not report to anyone. The CEO wishes to pick a subset of employees to attend a publicity event so that, for each employee, either that employee or to whom that employee reports must attend the event. However, no employee wants to attend the event, and they are only willing to go if they are paid extra. Each employee has a price (\$) they are willing to be paid in order to attend the event (including the CEO). Your task is to select a subset of employees to attend the event so that, for any employee, either that employee attends or the employee to whom they report attends (possibly both), while paying the minimum total dollars.

Formally, let T be a rooted binary tree with n nodes that represents the corporate structure, where the CEO correspond to the root node, $c \in T$. Each employee (node) $v \in T$ stores the price $p(v) \geq 0$ that the employee is willing to accept to attend the event, and the list $\ell(v)$ of $r(v)$ employees (nodes) that report directly to employee v . Any employee v without other employees that report to them has $r(v) = 0$ and empty list $\ell(v)$. Describe an efficient dynamic programming algorithm that computes a suitable subset of employees to attend the event that has minimum total price. The target runtime is $O(n)$. [Hint: Compare and contrast this problem with the maximum independent set (MIS) problem from lecture and our textbook. Don't aim for the target runtime from the start. First obtain a **correct** solution, and **then** consider how to improve the runtime.]

Solution:

MaxAttend(v): given node v , compute the minimum price of a suitable subset of employees to attend the event.

Input: a list of nodes v representing every employee with list of those employee who report to them $\ell(v)$ and the price it takes to include employee v $P(v)$.

Output: attend(c), which is the minimum amount of paying that satisfies condition.

$$\text{MaxAttend}(v) = \min \begin{cases} \text{attend}_y(v) = P(v) + \min\{\sum_{u \in \ell(v)} \text{attend}_y(u), \sum_{u \in \ell(v)} \text{attend}_n(u)\} \\ \text{attend}_n(v) = \sum_{u \in \ell(v)} \text{attend}_y(u) \end{cases}$$

Memoization DS: We use the tree structure itself, adding attend_y and attend_n to each node.

Evaluation Order: Post-order traversal of the tree.

Runtime Analysis:

For each node, we perform a constant amount of work plus work equal to the number of children (max is 2 since we have a binary tree). The total number of child in a tree is $n-1$ (where n is the number of nodes), so the total work is $O(n)$.

Solving the problem: Call MaxAttend(c), where c is ceo node.

Problem 4

Freddie is the owner of Freddie's Patterned Mousepads (FPM), the premier vendor on Etsy for custom-sized neoprene mousepads. No matter how small or how large a mousepad you need (in whole units), FPM has you covered. Every one of their mousepads features the same world-renowned signature pattern: down-sloping stripes in Duke Navy Blue (hex code #012169) on solid grey.

To make the mousepads, Freddie begins with a single large rectangular $a \times b$ neoprene sheet, for some integers $a, b \geq 1$. Using his cutting machine, he can make either a vertical or horizontal cut across the sheet, dividing it into two smaller rectangular sheets. After a sequence of cuts, he has a pile of various rectangular sheets, which he finishes into mousepads by stitching up the sides then posting them for sale on his Etsy store. However, the prices his customers are willing to pay for different sized mousepads seem to be completely arbitrary; for example, a 1×7 mousepad may sell for \$37 and a 2×7 mousepad may only sell for \$4. In particular, because of the pattern, a 1×7 mousepad and 7×1 mousepad look different and may sell for wildly different prices. Overwhelmed, Freddie is asking for your help to figure out the most money he can make from a single $a \times b$ neoprene sheet.

A template file, `mousepads.py`, is provided on Canvas. Design and implement an **iterative dynamic programming** algorithm as `MAXREVENUE(a, b, P[0..a][0..b])` in `mousepads.py` that solves Freddie's problem, where the first two parameters a, b denote the $a \times b$ dimensions of the initial neoprene sheet and $P[0..a][0..b]$ is a 2D array such that $P[i, j]$ stores the price for which an $i \times j$ mousepad will sell, for any integers $0 \leq i \leq a$ and $1 \leq j \leq b$. P is such that $P[0][j] = P[i][0] = 0$ for all i, j —Freddie cannot sell a mousepad with zero width or height! For full credit, the running time of your algorithm should be within constant factors of a reference solution with asymptotic runtime $O(ab(a + b))$.

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 12 & 13 & 3 & 7 \\ 0 & 8 & 27 & 30 & 11 & 9 \\ 0 & 8 & 7 & 0 & 8 & 16 \end{bmatrix}$$